Programación I

Estructuras de Decisión (Condicionales)

Aprenderemos cómo lograr que nuestros programas dejen de ser simples listas de pasos lineales y empiecen a 'pensar' y tomar diferentes caminos según los datos que reciban. Veremos la lógica detrás de estas decisiones y cómo implementarlas en C++.

Introducción

Las estructuras de decisión o condicionales son fundamentales en programación ya que permiten que un programa tome decisiones en tiempo de ejecución. Sin ellas, todos los programas serían secuenciales y no podrían adaptarse a diferentes entradas o situaciones.

Una condición se establece mediante una proposición lógica. La misma puede tener uno de dos posibles resultados: **verdadero** o **falso**.

Las palabras reservadas para representar verdadero y falso en C++ son:

- ✓ verdadero → **true**
- ✓ falso → **false**

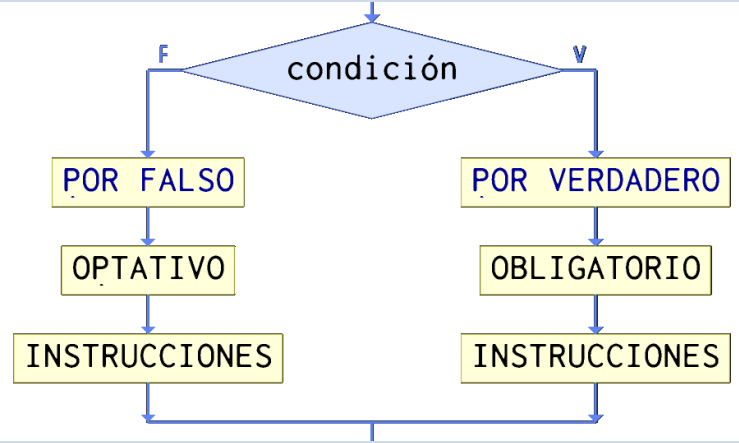
Palabras Reservadas en C++ para Decisiones

En C++ tenemos un 'vocabulario' específico para manejar decisiones. Las dos grandes estructuras son el **if**, que se usa para decisiones basadas en condiciones lógicas, y el **switch**, que es ideal cuando tenemos una sola variable que puede tomar muchos valores fijos (como un menú).

Palabra	Uso	Ejemplo
if	Inicia una estructura de decisión	if (x > 0) { ... }
else	Define el camino alternativo (opcional)	else { ... }
switch	Inicia una estructura de decisión múltiple	switch (opcion) { ... }
case	Define un caso dentro de un switch	case 'A':
default	Define el caso por defecto en switch	default: ...
break	Sale de una estructura switch (o bucle)	break;

Modelo genérico

Aquí tenemos el modelo genérico de una decisión. Todo empieza con una pregunta (la condición) encerrada entre paréntesis. Si la respuesta es **true**, entramos sí o sí al primer bloque de llaves. El bloque **else** es el plan B: solo se ejecuta si la respuesta a nuestra pregunta inicial fue **false**. Recuerden: el **if** puede vivir solo, pero el **else** siempre necesita un **if** previo."



Diagrama

```
if (condición) {
      /* POR VERDADERO
      OBLIGATORIO: Instrucciones si
      la  condición es verdadera */
}
else {
      /* POR FALSO
      OPTATIVO: Instrucciones si
      la  condición es falsa */
}
```

Código C++

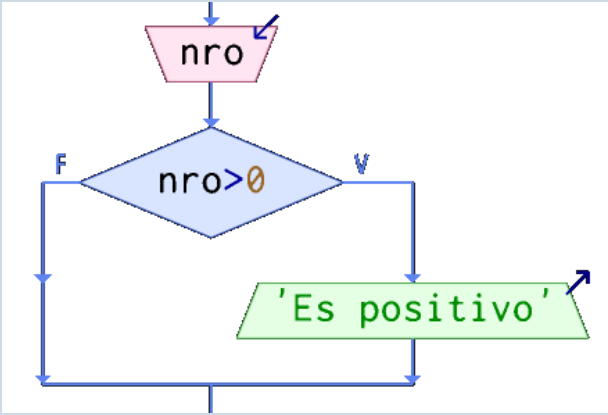
Operadores Relacionales

Para que el programa tome una decisión, primero debe realizar una comparación. Los **operadores relacionales** son las herramientas que usamos para esto. Un punto crucial: en C++ usamos el doble igual (**==**) para preguntar si algo es igual a otra cosa, ya que el igual simple (**=**) lo reservamos para guardar valores en variables. Fíjense en los ejemplos con N = 10: si pregunto si 10 es mayor o igual a 10, la respuesta es **true**, porque cumple una de las dos condiciones.

Operador	Significado	Ejemplo (si N = 10)	Resultado
==	Igual a	N == 10	**true**
!=	Distinto de	N != 10	**false**
>	Mayor que	N > 0	**true**
<	Menor que	N < 10	**false**
>=	Mayor o igual que	N >= 10	**true**
<=	Menor o igual que	N <= 5	**false**

Decisión Simple

La **decisión simple** es una estructura de control que condiciona la ejecución de un bloque de código al cumplimiento de una **proposición lógica**. Si la condición es **true**, el flujo se desvía para ejecutar la instrucción; si es **false**, el programa simplemente la omite y continúa su curso lineal.



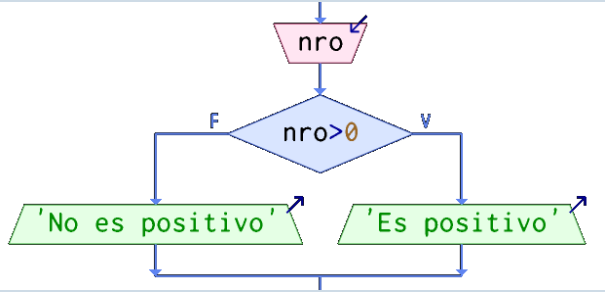
Diagrama

```
int nro;
cin >> nro;
if (nro > 0) {
    cout << "Es positivo";
}
```

Código C++

Decisión Doble

Con una **decisión doble** el programa funciona de la siguiente manera: llega a un punto donde debe elegir entre dos caminos. El **if** ejecuta la salida por el verdadero si la condición es verdadera, mientras que el **else** actúa como el complemento lógico, el 'plan B'. Esto garantiza que el algoritmo siempre tenga una respuesta preparada, ya sea que la proposición se cumpla o no."

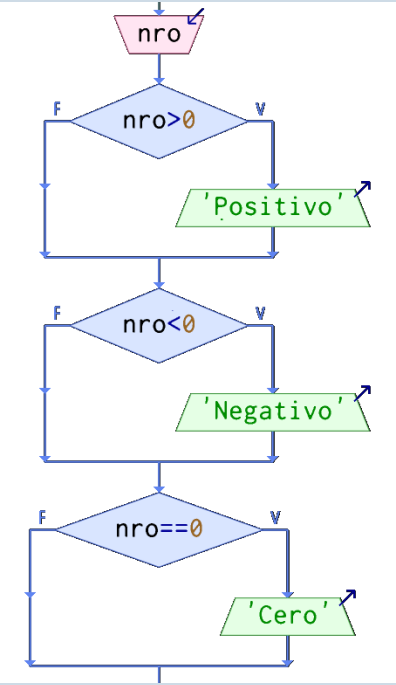


Diagrama

```
int nro;
if (nro > 0) {
    cout << "Positivo";
}
else {
    cout << "No es positivo";
}
```

Código C++

Decisión Secuencial



Diagrama

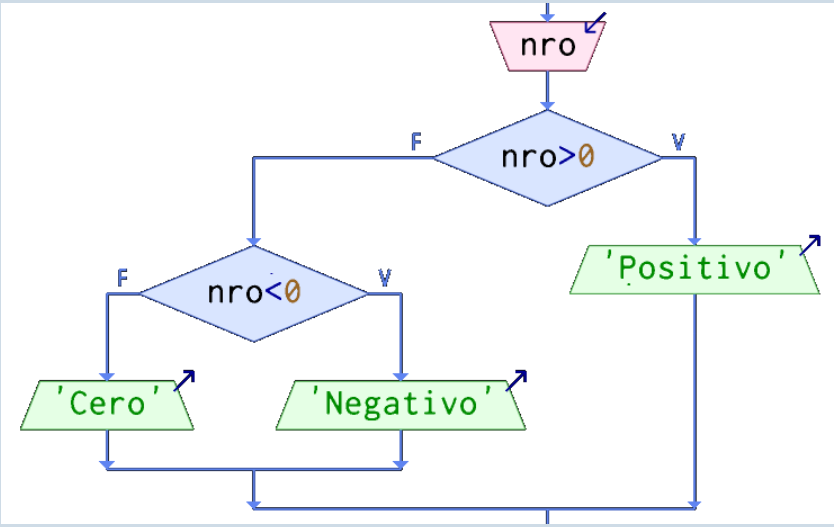
La **decisión secuencial** consiste en una serie de proposiciones lógicas independientes. El programa está obligado a evaluar todas y cada una de las condiciones (**if**), sin importar si alguna de las anteriores ya fue verdadera.

```
int nro;
if (nro > 0) {
    cout << "Positivo";
}
if (nro < 0) {
    cout << “Negativo";
}
if (nro == 0) {
    cout << “Cero";
}
```

Código C++

Decisión Anidada

Una estructura de decisión dentro de otra.



Diagrama

```
int nro;
if (nro > 0) {
    cout << "Positivo";
}
else {
    if (nro < 0) {
        cout << "Negativo";
    }
    else {
        cout << "Cero";
    }
}
```

Código C++

Errores comunes

- ✓ **Confusión de Operadores (= vs ==):** Usar el de asignación para comparar. Es el error más difícil de detectar porque el código compila, pero la lógica falla.

- ✓ **El Punto y Coma Traicionero:** Poner ; justo después del if (...). Esto anula la condición y hace que el bloque de abajo se ejecute siempre.

- ✓ **Omisión de Llaves ({ }):** Olvidarlas cuando el if debe controlar más de una línea. Sin ellas, solo la primera instrucción queda protegida por la condición.

- ✓ **Rangos Matemáticos Incorrectos:** Escribir if (10 < x < 20). En C++ esto es un error lógico; la forma correcta es usar el AND: if (x > 10 && x < 20).